YZ 203 - Week 5: Lab / Practice Session Notes

Lab Objectives

This lab gives the basket a player. It starts with a single key press and ends with a playable catch-the-egg game. Work through the exercises in order; each one adds one idea.

By the end of this session, the student will be able to:

- Handle keyboard input with `KeyListener` and `KeyAdapter`, and explain why `KeyListener` has no lambda form
- Explain and fix the keyboard focus problem
- Move an object smoothly: key listeners set flags, the Timer applies them
- Use key bindings (`InputMap`, `ActionMap`) as an alternative that depends much less on the focus
- Handle mouse movement and clicks with `MouseMotionAdapter` and `MouseAdapter`
- Detect a collision with `Rectangle.intersects()` and count a catch exactly once
- Combine everything into Falling Egg v2: basket, egg, score, missed counter, Pause and a speed slider

Level	Exercises
Easy	1. Key echo, 2. Arrow keys (jerky), 3. The focus problem
Medium	4. Smooth movement, 5. Key bindings, 6. Mouse follow, 7. Mouse clicks, 8. Hit test
Hard	9. Basket class, 10. Catch the egg, 11. Falling Egg v2
Challenge	12. Keyboard or mouse control

Duration: about 2 class hours for the Core exercises; the rest is homework. **Required materials:** JDK and NetBeans from Week 1.

Imports used in this lab: `javax.swing.*`, `java.awt.*` and `java.awt.event.*`.

How to Work Through This Lab

One package per exercise. Several exercises define their own `Egg` and `Basket` classes. If you put every exercise in the same package, NetBeans reports "duplicate class". So create a new package for each exercise (for example `week5.ex01`, `week5.ex02`, ..., `week5.ex12`) and create the class inside it. NetBeans adds the `package week5.exNN;` line at the top of the file for you. For Exercises 11 and 12, copy the finished files of the previous exercise into the new package and edit them there.

Core and extra exercises. There are 12 exercises, and nobody is expected to finish all of them in class. Work in order. Whatever you do not finish becomes homework.

Group	Exercises	Why
Core (aim to finish in class)	1 Key echo, 4 Smooth movement, 6 Mouse follow and drag, 8 Hit test, 10 Catch the egg, 11 Falling Egg v2	They carry the main ideas of the week and build the game
Extra (finish at home if time runs out)	2 Jerky keys, 3 Focus problem, 5 Key bindings, 7 Mouse clicks, 9 Basket class	They deepen one idea each; 2 and 3 explain why 4 is designed the way it is
Challenge	12 Keyboard or mouse control	Optional

Predict, run, explain. Many exercises end with a "Try it" or a "Predict first" question. Write down your prediction before you run the code, then run it, then explain any difference. That habit teaches more than copying the answer.

TODO exercises. In Exercises 4, 7, 8 and 12 part of the code is left for you to write. The program compiles and runs without it, but does not yet do what the expected output shows.

Exercise 1 (Easy): Key Echo

Goal: Receive your first keyboard event and show which key was pressed.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

```
class KeyPanel extends JPanel {
    private String text = "Press a key";

    KeyPanel() {
        setFocusable(true);
        addKeyListener(new KeyListener() {
            public void keyPressed(KeyEvent e) {
                text = "Pressed: " + KeyEvent.getKeyText(e.getKeyCode());
                repaint();
            }
            public void keyReleased(KeyEvent e) { }
            public void keyTyped(KeyEvent e) { }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setFont(new Font("Arial", Font.BOLD, 22));
        g.drawString(text, 20, 50);
    }
}

public class KeyEcho {
    public static void main(String[] args) {
        JFrame f = new JFrame("Key Echo");
        KeyPanel p = new KeyPanel();
        f.add(p);
        f.setSize(300, 140);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

Step by step:

1. `KeyListener` is an interface with **three** methods, so we must supply all three, even if two stay empty.
2. `new KeyListener() { ... }` is an anonymous class: a class without a name, created on the spot. A lambda is not possible here because there is more than one method.

3. `e.getKeyCode()` returns a number such as `KeyEvent.VK_SPACE`; `KeyEvent.getKeyText` turns it into readable text.
4. `setFocusable(true)` and `requestFocusInWindow()` make the panel the receiver of keys. Without them, nothing would be shown. Exercise 3 explains why.

Expected output:



Try it: press Shift, Enter, F1 and an arrow key. Which keys produce a name? Then replace `getKeyText(e.getKeyCode())` with `"" + e.getKeyChar()` and compare what Shift and the arrow keys show.

Exercise 2 (Easy): Arrow Keys Move a Square (The Simple Way)

Goal: Use `KeyAdapter` so that only the one method we need is written. Then observe why this version does not feel like a game.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MovePanel extends JPanel {
    private int x = 100;

    MovePanel() {
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_LEFT)  x -= 10;
                if (e.getKeyCode() == KeyEvent.VK_RIGHT) x += 10;
                repaint();
            }
        });
    }

    @Override
```

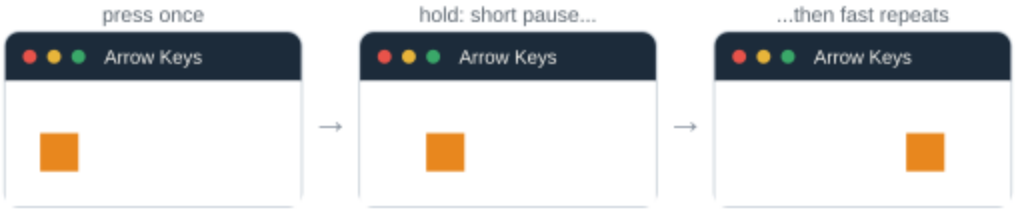
```
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

public class MoveJerky {
    public static void main(String[] args) {
        JFrame f = new JFrame("Arrow Keys");
        MovePanel p = new MovePanel();
        f.add(p);
        f.setSize(360, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

Step by step:

1. `KeyAdapter` implements all three `KeyListener` methods with empty bodies. We extend it and override only `keyPressed`.
2. Each key press moves the square 10 pixels and asks for a repaint.
3. Hold the left arrow for two seconds: the square moves once, pauses, then moves quickly. That pause and the fast repeat are the operating system's key repeat, not our code.

Expected output:



Try it: hold the key and watch the pause. Then change `10` to `3`. Does the movement feel smoother, or only slower? Exercise 4 fixes the real problem.

Exercise 3 (Easy to Medium): The Focus Problem

Goal: Meet the most common keyboard bug on purpose, then fix it. Key events go only to the component that has the keyboard focus. Clicking a button gives the focus to that button.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class FocusPanel extends JPanel {
    private int x = 100;

    FocusPanel() {
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_LEFT)  x -= 10;
                if (e.getKeyCode() == KeyEvent.VK_RIGHT) x += 10;
                repaint();
            }
        });
    }

    void reset() { x = 100; repaint(); }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.fillRect(x, 60, 40, 40);
    }
}

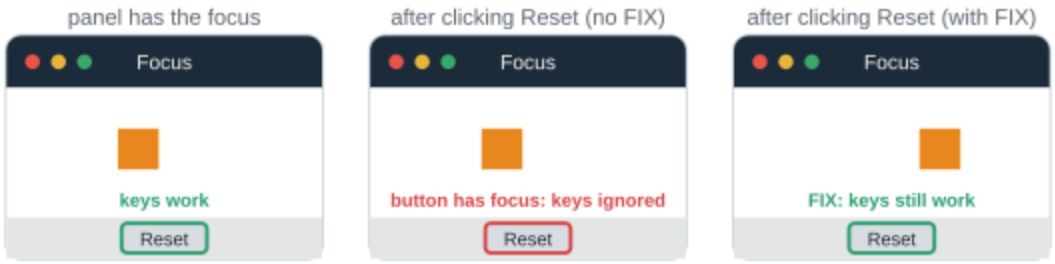
public class FocusDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Focus");
        FocusPanel p = new FocusPanel();
        JButton reset = new JButton("Reset");
        reset.setFocusable(false);   // FIX: the button never steals focus
        reset.addActionListener(e -> p.reset());
        f.add(p);
        f.add(reset, BorderLayout.SOUTH);
```

```
        f.setSize(320, 220);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

Step by step:

1. First run the program **with the FIX line commented out**. Move the square with the arrows, click Reset, then press the arrows again. Nothing moves: the button now has the focus, so the key events go to it.
2. Fix A (shown): `reset.setFocusable(false)`. The button can still be clicked, but it never takes the focus.
3. Fix B: remove the FIX line and instead call `p.requestFocusInWindow()` at the end of the button's listener to give the focus back.
4. `setFocusable(true)` on the panel is also needed: panels do not accept the focus by default.

Expected output:



Try it: use Fix B instead of Fix A. Click Reset, then try the arrows. Then add a `JSlider` below the panel and notice that it steals the focus too: every control you add needs the same treatment.

Exercise 4 (Medium): Smooth Movement with Flags and a Timer

Goal: Fix the jerky movement of Exercise 2. Key listeners only record which keys are down; the Timer moves the square on a steady beat. The Timer and the drawing are written for you. **You write the two listener methods** (TODO 1 and TODO 2). The program runs now, but the square does not move until you finish them.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class SmoothPanel extends JPanel {
    private int x = 100;
    private boolean left, right;

    SmoothPanel() {
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                // TODO 1: when LEFT is pressed set left = true,
                //         when RIGHT is pressed set right = true
            }
            @Override
            public void keyReleased(KeyEvent e) {
                // TODO 2: when LEFT is released set left = false,
                //         when RIGHT is released set right = false
            }
        });
        new Timer(16, e -> {
            if (left)  x -= 5;
            if (right) x += 5;
            x = Math.max(0, Math.min(x, getWidth() - 40));
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

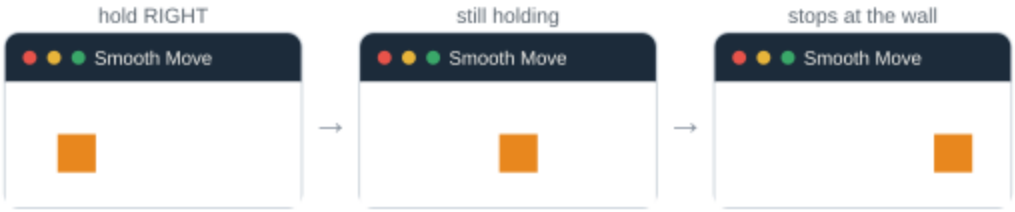
public class MoveSmooth {
    public static void main(String[] args) {
        JFrame f = new JFrame("Smooth Move");
        SmoothPanel p = new SmoothPanel();
        f.add(p);
```

```
        f.setSize(360, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

Step by step:

1. The listeners must not move anything. They only write down which keys are held: `keyPressed` sets a flag to `true`, `keyReleased` sets it back to `false`. Compare with the `if` lines in Exercise 2, which used `e.getKeyCode() == KeyEvent.VK_LEFT`.
2. The Timer reads the flags every 16 ms and moves the square 5 pixels. The speed is chosen by us, not by the operating system's key repeat.
3. `Math.max(0, Math.min(x, getWidth() - 40))` clamps the square inside the panel: never below 0 and never past the right edge.
4. When you are done, holding both arrows at once should make the square stand still, because the two moves cancel out.

Expected output:



Predict first, then run: (a) What happens when LEFT and RIGHT are held together? (b) What happens after you let go of a key if you delete the `keyReleased` method? (c) Press LEFT, switch to another window with Alt+Tab, release LEFT there, and come back. What do you see, and why? (Exercise 11 fixes this.)

Exercise 5 (Medium): Key Bindings

Goal: Attack the focus problem from another side. A key binding maps a keystroke to a named action, and Swing looks it up while the window is active, even if a button has the focus. It depends much less on which component has the focus than a KeyListener does.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class BindPanel extends JPanel {
    private int x = 100;

    BindPanel() {
        InputMap im = getInputMap(WHEN_IN_FOCUSED_WINDOW);
        ActionMap am = getActionMap();
        im.put(KeyStroke.getKeyStroke("LEFT"), "left");
        im.put(KeyStroke.getKeyStroke("RIGHT"), "right");
        am.put("left", new AbstractAction() {
            public void actionPerformed(ActionEvent e) { x -= 10; repaint();
}
        });
        am.put("right", new AbstractAction() {
            public void actionPerformed(ActionEvent e) { x += 10; repaint();
}
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(x, 80, 40, 40);
    }
}

public class MoveBindings {
    public static void main(String[] args) {
        JFrame f = new JFrame("Key Bindings");
        f.add(new BindPanel());
        f.add(new JButton("Click me"), BorderLayout.SOUTH);
        f.setSize(360, 220);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `InputMap` maps a keystroke to a name; `ActionMap` maps that name to an `Action`. The name in the middle makes the mapping easy to change later.
2. `AbstractAction` is a ready-made class that implements `Action`, which extends `ActionListener`. It is the Week 1 interface idea again.
3. `WHEN_IN_FOCUSED_WINDOW` means the binding works while the window is active, even if a button has the focus. A slider or a text field that has the focus may still consume the arrow keys itself, so such controls still win.
4. There is no `setFocusable` and no `requestFocusInWindow`. Click the button and the arrows still work.

Expected output:



Try it: make the movement smooth, as in Exercise 4. Replace the two keystrokes with `"pressed LEFT"`, `"released LEFT"`, `"pressed RIGHT"` and `"released RIGHT"`, bind four actions that set and clear two flags, and add the Timer from Exercise 4.

Exercise 6 (Medium): A Circle That Follows the Mouse

Goal: Handle mouse movement. Unlike the keyboard, the mouse needs no focus: events go to the component under the pointer.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class FollowPanel extends JPanel {
    private int mx = 100, my = 60;

    FollowPanel() {
        addMouseMotionListener(new MouseMotionAdapter() {
            @Override
            public void mouseMoved(MouseEvent e) {
                mx = e.getX();
                my = e.getY();
```

```
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillOval(mx - 15, my - 15, 30, 30);
    }
}

public class MouseFollow {
    public static void main(String[] args) {
        JFrame f = new JFrame("Mouse Follow");
        f.add(new FollowPanel());
        f.setSize(300, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `MouseMotionListener` has two methods (`mouseMoved`, `mouseDragged`), so we use `MouseMotionAdapter` and override only the one we need.
2. `e.getX()` and `e.getY()` are relative to the component that has the listener, here the panel.
3. `fillOval` expects the top-left corner of the box, so we subtract half the size (15) to center the circle on the pointer.
4. As in every game panel: the listener only records numbers; `paintComponent` draws.

Expected output:



Predict first, then run: (a) Why do we subtract 15 from `mx` and `my`? What do you see if you do not? (b) Change `mouseMoved` to `mouseDragged`. When does the circle move now? When would each be the right choice in a game?

Part B: Drag and Drop a Rectangle

Goal: Use three mouse events together. `mousePressed` picks the rectangle up, `mouseDragged` moves it, `mouseReleased` drops it.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class DragPanel extends JPanel {
    private final Rectangle box = new Rectangle(60, 50, 80, 50);
    private int offX, offY;
    private boolean dragging = false;

    DragPanel() {
        MouseAdapter ma = new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                if (box.contains(e.getPoint())) {
                    dragging = true;
                    offX = e.getX() - box.x;
                    offY = e.getY() - box.y;
                    repaint();
                }
            }
            @Override
            public void mouseDragged(MouseEvent e) {
                if (dragging) {
                    box.setLocation(e.getX() - offX, e.getY() - offY);
                    repaint();
                }
            }
            @Override
            public void mouseReleased(MouseEvent e) {
                dragging = false;
                repaint();
            }
        };
```

```
        addMouseListener(ma);
        addMouseMotionListener(ma);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(dragging ? Color.RED : Color.ORANGE);
        g.fillRect(box.x, box.y, box.width, box.height);
    }
}

public class MouseDrag {
    public static void main(String[] args) {
        JFrame f = new JFrame("Drag a Rectangle");
        f.add(new DragPanel());
        f.setSize(360, 240);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `MouseAdapter` implements both `MouseListener` and `MouseMotionListener`, so one object can be registered twice: `addMouseListener(ma)` and `addMouseMotionListener(ma)`.
2. `box.contains(e.getPoint())` is true only if the press landed on the rectangle. A press elsewhere does not start a drag.
3. `offX` and `offY` remember where inside the rectangle you grabbed it, so it does not jump to the pointer.
4. `dragging` is `false`, then `true` between the press and the release. `mouseDragged` does nothing unless it is `true`.

Expected output:



Predict first, then run: What happens if you remove `offX` and `offY` and always move the rectangle's top-left corner to the pointer? Then press on the empty area and drag: why does nothing move?

Exercise 7 (Medium): Mouse Clicks, Buttons and a Counter

Goal: React to clicks, tell the left and right buttons apart, and keep a counter in a private field. The panel and the counter are written for you. **You write the two TODOs inside `mousePressed`.** Until you do, the circle stays where it is and does not change color.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class ClickPanel extends JPanel {
    private int x = 100, y = 60, clicks = 0;
    private Color color = Color.ORANGE;

    ClickPanel() {
        addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                clicks++;
                // TODO 1: store the click position in x and y
                //         (use e.getX() and e.getY())
                // TODO 2: set color to Color.BLUE for the right button
                //         and Color.ORANGE otherwise
                //         (hint: SwingUtilities.isRightMouseButton(e))
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(color);
        g.fillOval(x - 20, y - 20, 40, 40);
        g.setColor(Color.DARK_GRAY);
        g.drawString("Clicks: " + clicks, 10, 20);
    }
}
```

```
public class MouseClick {
    public static void main(String[] args) {
        JFrame f = new JFrame("Mouse Click");
        f.add(new ClickPanel());
        f.setSize(300, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `MouseListener` has five methods, so `MouseAdapter` saves us from writing four empty ones.
2. `mousePressed` fires as soon as the button goes down; `mouseClicked` fires only after a press and a release without moving in between. Games usually prefer `mousePressed`.
3. `SwingUtilities.isRightMouseButton(e)` is a static method: it needs no object, just the event.
4. `x`, `y`, `color` and `clicks` are private fields. The listener changes them; `paintComponent` only reads them. Test: the counter already works before you write the TODOs.

Expected output:



Try it: make the middle button draw a green circle. (Hint: `SwingUtilities.isMiddleMouseButton(e)`.) Then add a second counter that counts only right clicks.

Exercise 8 (Medium): Hit Test with Rectangle.intersects()

Goal: Answer the question every game asks: do these two boxes overlap? The drawing and the two rectangles are written for you. **You write TODO 1 (follow the mouse) and TODO 2 (the color rule).** Until then the green box stays in the corner and never turns red.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class HitPanel extends JPanel {
    private final Rectangle target = new Rectangle(120, 70, 100, 60);
    private final Rectangle mover = new Rectangle(0, 0, 50, 40);

    HitPanel() {
        addMouseMotionListener(new MouseMotionAdapter() {
            @Override
            public void mouseMoved(MouseEvent e) {
                // TODO 1: move 'mover' so that the pointer is at its center
                //         (hint: mover.setLocation(...); the box is 50 x 40)
                repaint();
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.GRAY);
        g.fillRect(target.x, target.y, target.width, target.height);
        g.setColor(Color.GREEN);
        // TODO 2: use mover.intersects(target) so that the box is drawn
        //         RED while it overlaps the target and GREEN otherwise
        g.fillRect(mover.x, mover.y, mover.width, mover.height);
    }
}

public class HitDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Hit Test");
        f.add(new HitPanel());
        f.setSize(340, 220);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `java.awt.Rectangle` stores `x`, `y`, `width` and `height` and already knows how to compare itself with another rectangle.
2. `a.intersects(b)` is `true` when the rectangles overlap by a positive area, both horizontally and vertically, and `false` otherwise. Two rectangles that only share an edge do not intersect.
3. A ternary `condition ? Color.RED : Color.GREEN` picks a color from a yes or no answer; an `if` statement works just as well.
4. This is a **bounding box** test, not an exact shape test. A real egg is an oval and a real basket is a curve, yet we compare the rectangles around them. For this game that is accurate enough.

Expected output:



Try it: move the mouse so the green box just touches the gray one. Does a shared edge count as overlapping? Then print `mover.intersection(target)` to the console while overlapping and describe what it returns.

Exercise 9 (Medium to Hard): The Basket Class, Moved with the Arrow Keys

Goal: Give the basket its own class, like `Egg`, and move it smoothly. The panel records the keys; the Timer moves the basket.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Basket {
    private int x;
    private final int y;
    private static final int W = 90, H = 30, STEP = 6;

    Basket(int x, int y) { this.x = x; this.y = y; }
```

```
    void moveLeft()  { x -= STEP; }
    void moveRight() { x += STEP; }

    void clamp(int fieldWidth) {
        x = Math.max(0, Math.min(x, fieldWidth - W));
    }

    Rectangle getBounds() { return new Rectangle(x, y, W, H); }

    void draw(Graphics g) {
        g.setColor(new Color(139, 90, 43));
        g.fillArc(x, y - H, W, 2 * H, 180, 180);
    }
}

class BasketPanel extends JPanel {
    private final Basket basket = new Basket(150, 300);
    private boolean left, right;

    BasketPanel() {
        setPreferredSize(new Dimension(400, 340));
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override public void keyPressed(KeyEvent e)  {
setKey(e.getKeyCode(), true); }
            @Override public void keyReleased(KeyEvent e) {
setKey(e.getKeyCode(), false); }
        });
        new Timer(16, e -> {
            if (left)  basket.moveLeft();
            if (right) basket.moveRight();
            basket.clamp(getWidth());
            repaint();
        }).start();
    }

    private void setKey(int code, boolean down) {
        if (code == KeyEvent.VK_LEFT)  left = down;
        if (code == KeyEvent.VK_RIGHT) right = down;
    }

    @Override
    protected void paintComponent(Graphics g) {
```

```
        super.paintComponent(g);
        basket.draw(g);
    }
}

public class BasketKeys {
    public static void main(String[] args) {
        JFrame f = new JFrame("Basket");
        BasketPanel p = new BasketPanel();
        f.add(p);
        f.pack();
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

Step by step:

1. `W`, `H` and `STEP` are `static final`: constants shared by every `Basket`. This is the right use of `static`.
2. `getBounds()` already exists for the next exercise: the basket reports its own box as a `Rectangle`. The box is the opening of the bowl, from `y` down to `y + H`.
3. The helper method `setKey` removes the duplicated `if` lines from the two listeners.
4. `BasketPanel` has a `Basket` (composition). It asks the basket to move, clamp and draw, and never touches `x` directly.

Expected output:



Try it: make `STEP` bigger and notice that the speed changes without touching any other line. Then add a second pair of keys, A and D, that also move the basket.

Exercise 10 (Hard): Catch the Egg

Goal: Add the rule that makes it a game. If the egg overlaps the basket, the player scores and a new egg starts. If the egg falls past the bottom, it counts as missed.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Egg {
    private int x, y;
    private final int speed;

    Egg(int x, int speed) { this.x = x; this.speed = speed; }

    boolean fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void respawn(int newX) { x = newX; y = 0; }

    Rectangle getBounds() { return new Rectangle(x, y, 30, 40); }

    void draw(Graphics g) {
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
        g.setColor(Color.WHITE);
        g.fillOval(x + 7, y + 8, 8, 10);
    }
}

class Basket {
    private int x;
    private final int y;
    private static final int W = 90, H = 30, STEP = 6;

    Basket(int x, int y) { this.x = x; this.y = y; }
    void moveLeft()  { x -= STEP; }
    void moveRight() { x += STEP; }
    void clamp(int fieldWidth) { x = Math.max(0, Math.min(x, fieldWidth -
W)); }
    Rectangle getBounds() { return new Rectangle(x, y, W, H); }

    void draw(Graphics g) {
        g.setColor(new Color(139, 90, 43));
        g.fillArc(x, y - H, W, 2 * H, 180, 180);
```

```
    }
}

class CatchPanel extends JPanel {
    private final Egg egg = new Egg(100, 4);
    private final Basket basket = new Basket(150, 320);
    private final java.util.Random rnd = new java.util.Random();
    private boolean left, right;
    private int score = 0, missed = 0;

    CatchPanel() {
        setPreferredSize(new Dimension(400, 360));
        setBackground(new Color(222, 240, 255));
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override public void keyPressed(KeyEvent e)  {
setKey(e.getKeyCode(), true); }
            @Override public void keyReleased(KeyEvent e) {
setKey(e.getKeyCode(), false); }
        });
        new Timer(16, e -> update()).start();
    }

    private void setKey(int code, boolean down) {
        if (code == KeyEvent.VK_LEFT)  left = down;
        if (code == KeyEvent.VK_RIGHT) right = down;
    }

    private void update() {
        if (left)  basket.moveLeft();
        if (right) basket.moveRight();
        basket.clamp(getWidth());

        boolean wrapped = egg.fall(getHeight());
        if (egg.getBounds().intersects(basket.getBounds())) {
            score++;
            respawn();
        } else if (wrapped) {
            missed++;
            respawn();
        }
        repaint();
    }
```

```
    private void respawn() {
        egg.respawn(rnd.nextInt(getWidth() - 30));
    }

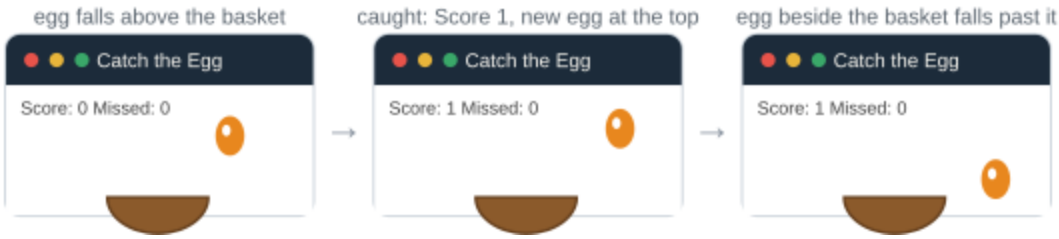
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
        basket.draw(g);
        g.setColor(Color.DARK_GRAY);
        g.drawString("Score: " + score + "   Missed: " + missed, 10, 20);
    }
}

public class CatchDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Catch the Egg");
        CatchPanel p = new CatchPanel();
        f.add(p);
        f.pack();
        f.setResizable(false);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        p.requestFocusInWindow();
    }
}
```

Step by step:

1. `update()` has a fixed order: apply the input, move the objects, check the rules, then request a repaint.
2. `egg.getBounds().intersects(basket.getBounds())` asks each object for its own box. The panel does not know how wide an egg is.
3. A caught egg overlaps the basket for many ticks in a row. Calling `respawn()` in the same tick that we count the catch is what prevents the score from jumping by 15 or more.
4. `else if (wrapped)`: a tick ends with at most one outcome, caught or missed.
5. `setResizable(false)` keeps the pixel positions valid, because the basket's `y` is fixed at 320.

Expected output:



Try it: delete `respawn();` from the catch branch and catch one egg. How much does the score jump, and why? Put it back.

Exercise 11 (Hard): Falling Egg v2

Goal: Extend the finished Exercise 10 into the playable game of the week: a Pause / Start button, a tick-delay slider, and a fix for stuck keys. **Copy your Exercise 10 files into a new package (`week5.ex11`) and start from them.** `Egg` and `Basket` do not change. Only the new and changed code is shown below.

Step 1. Changes in `CatchPanel`. Rename it to `GamePanel` and make three changes: the Timer becomes a field, a focus listener resets the keys, and two public methods let the window control the Timer.

```
// 1. new field, next to the other fields
private final Timer timer;

// 2. in the constructor, replace the old line  new Timer(16, e ->
update()).start();
//    by these:
addFocusListener(new FocusAdapter() {
    @Override public void focusLost(FocusEvent e) { left = false; right =
false; }
});
timer = new Timer(16, e -> update());
timer.start();

// 3. two new methods, anywhere in the class
boolean toggle() {
    if (timer.isRunning()) timer.stop(); else timer.start();
    return timer.isRunning();
}

void setSpeed(int delay) { timer.setDelay(delay); }
```

Step 2. Replace `main`. The window now holds the game, a button and a slider.

```
public class FallingEggV2 {
    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg v2");
        GamePanel game = new GamePanel();
        JButton toggle = new JButton("Pause");
        JSlider speed = new JSlider(5, 40, 16);
        toggle.setFocusable(false);
        speed.setFocusable(false);

        JPanel bar = new JPanel();
        bar.add(toggle);
        bar.add(new JLabel("Tick delay:"));
        bar.add(speed);

        toggle.addActionListener(e ->
            toggle.setText(game.toggle() ? "Pause" : "Start"));
        speed.addChangeListener(e -> game.setSpeed(speed.getValue()));

        f.add(game);
        f.add(bar, BorderLayout.SOUTH);
        f.pack();
        f.setResizable(false);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        game.requestFocusInWindow();
    }
}
```

Step by step:

1. `GamePanel` is a `JPanel` (inheritance) that has an `Egg`, a `Basket` and a `Timer` (composition). Its public doors are only `toggle()` and `setSpeed()`.
2. `toggle.setFocusable(false)` and `speed.setFocusable(false)` are the fix from Exercise 3. Remove them and watch the keys stop working after you click Pause.
3. While the game is paused the Timer is stopped, so the flags are not applied and the basket cannot move. Resume with the button.
4. The `focusLost` listener fixes the problem you may have seen in Exercise 4: if the window loses the focus while a key is held, `keyReleased` never arrives and the flag would stay `true`. Clearing both flags when the focus is lost is the cure.

5. The slider is called a delay slider on purpose. `timer.setDelay(...)` changes the time between ticks, and **everything** in the game updates on that tick: the egg, the basket and the collision check.

Expected output:



Predict first, then run: (a) Does changing the tick delay affect only the egg? What else becomes faster or slower? (b) Hold LEFT, press Alt+Tab, release LEFT in the other window and come back. Is the basket still moving? Remove the `focusLost` listener and try again. (c) Make the game harder: after every 5 catches the egg should fall faster, without changing the basket's speed. Which class should change, and does `GamePanel` need to know the number 5?

Exercise 12 (Challenge): Keyboard or Mouse, Chosen by a JComboBox

Goal: Let the player choose the control scheme. A `JComboBox` selects Keyboard or Mouse; in mouse mode the basket follows the pointer. The skeleton compiles and runs, but only the keyboard works until you finish the two TODOs.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Basket {
    private int x;
    private final int y;
    private static final int W = 90, H = 30, STEP = 6;

    Basket(int x, int y) { this.x = x; this.y = y; }
    void moveLeft()  { x -= STEP; }
```

```
    void moveRight() { x += STEP; }
    void centerAt(int cx) { x = cx - W / 2; }
    void clamp(int fieldWidth) { x = Math.max(0, Math.min(x, fieldWidth -
W)); }

    void draw(Graphics g) {
        g.setColor(new Color(139, 90, 43));
        g.fillArc(x, y - H, W, 2 * H, 180, 180);
    }
}

class ControlPanel extends JPanel {
    private final Basket basket = new Basket(150, 300);
    private boolean left, right, mouseMode = false;
    private int mouseX = 200;

    ControlPanel() {
        setPreferredSize(new Dimension(400, 340));
        setFocusable(true);
        addKeyListener(new KeyAdapter() {
            @Override public void keyPressed(KeyEvent e)  {
setKey(e.getKeyCode(), true); }
            @Override public void keyReleased(KeyEvent e) {
setKey(e.getKeyCode(), false); }
        });
        // TODO 1: add a MouseMotionAdapter whose mouseMoved only records
        //         the pointer: mouseX = e.getX()  (no moving here!)
        new Timer(16, e -> {
            // TODO 2: if mouseMode is true, call basket.centerAt(mouseX);
            //         otherwise apply the left and right flags below
            if (left)  basket.moveLeft();
            if (right) basket.moveRight();
            basket.clamp(getWidth());
            repaint();
        }).start();
    }

    private void setKey(int code, boolean down) {
        if (code == KeyEvent.VK_LEFT)  left = down;
        if (code == KeyEvent.VK_RIGHT) right = down;
    }

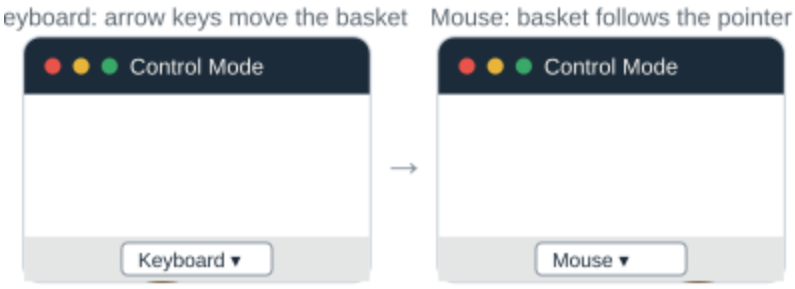
    void setMouseMode(boolean on) { mouseMode = on; }
```

```
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        basket.draw(g);
    }
}

public class ControlMode {
    public static void main(String[] args) {
        JFrame f = new JFrame("Control Mode");
        ControlPanel panel = new ControlPanel();
        JComboBox<String> mode = new JComboBox<>(new String[] {"Keyboard",
"Mouse"});
        mode.setFocusable(false);
        mode.addActionListener(e ->
panel.setMouseMode(mode.getSelectedIndex() == 1));

        f.add(panel);
        f.add(mode, BorderLayout.SOUTH);
        f.pack();
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        panel.requestFocusInWindow();
    }
}
```

Expected output once both TODOs are done:



Hints:

1. The mouse listener goes on the panel, the same component that draws, so `e.getX()` is already in the panel's coordinates.

2. Keep to the architecture of the week: the **listener only records** `mouseX`; the **Timer moves** the basket with `centerAt(mouseX)`, and `clamp` keeps it inside. Both input modes then go through the same update step.
3. In the Timer, an `if (mouseMode) { ... } else { ... }` around the movement is the whole second TODO.
4. Think ahead: how would you add a third mode, for example WASD keys? Which parts of the code would change, and which would not?

Behind the Scenes: The Timer Across Three Weeks, and the OOP Ideas

The line `new Timer(16, e -> ...)` has not changed since Week 3. Its listener has grown.

	Week 3	Week 4	Week 5
What the listener does	moves a component	updates numbers, calls `repaint()`	applies input, moves objects, checks catches, updates the score, calls `repaint()`
Who changes the state	the Timer	the Timer	the Timer; in a game panel, key and mouse listeners only record input
Who draws	`JLabel` components	`paintComponent`	`paintComponent`
Other event sources	button, slider	button, slider	keys, mouse, button, slider

In a game panel: one writer for the game state (the Timer) and listeners that only leave notes. That rule keeps a growing game predictable. In the small demos of Exercises 5 to 8 a listener changes its own fields directly, which is fine at that size.

OOP idea	Where you saw it
Interface	`KeyListener` has three methods, so it cannot take a lambda. `ActionListener` and `ChangeListener` have one (Exercises 1, 11).

OOP idea	Where you saw it
Inheritance	`KeyAdapter` implements all three methods with empty bodies; our class extends it and overrides one (Exercises 2 to 4).
Anonymous class	`new KeyAdapter() { ... }` creates a nameless subclass and one object of it in a single expression.
Callback	We never call `keyPressed`, `mouseMoved` or `paintComponent`. Swing calls them.
Encapsulation	`Basket` and `Egg` keep their coordinates private; `getBounds()` returns a fresh `Rectangle`, so nobody can change the object through it (Exercises 9, 10).
Composition	`GamePanel` has an `Egg`, a `Basket` and a `Timer`. It is none of them (Exercise 11).
static	`W`, `H` and `STEP` in `Basket` are `static final` constants. `SwingUtilities.isRightMouseButton(e)` is a static method (Exercises 7, 9).
Focus is shared state	Only one component receives key events. That is why controls call `setFocusable(false)`, or why bindings, which depend much less on the focus, are used instead (Exercises 3, 5). A lost focus can also leave a key stuck, which a focusLost listener fixes (Exercises 4, 11).

Homework / Research Questions

No answers are given here. Work them out and bring them to the next session. Any exercise you did not finish in class is also homework; the checklist below says which ones are required.

1. Build it. Extend Exercise 11. Level 1 is required, levels 2 and 3 are optional for those who want more.

1. **Required: lives and Game Over.** The player starts with 3 lives. Each missed egg costs one life. When the lives reach zero, stop the Timer and draw "Game Over" in the middle of the field. Which class should own the number of lives, `GamePanel` or something else, and why?
2. **Optional: restart.** Pressing Enter restarts the game with 3 lives. Should Enter be handled with a `KeyListener` or with a key binding, and why?
3. **Optional: more control.** Add a `JCheckBox` labeled "Show hit boxes" that draws the rectangles of the egg and the basket as outlines (`drawRect`), and a `JSpinner` (range 1 to 5) for the number of starting lives. Which class should draw its own hit box, and what do you have to add to it?

2. Investigate. In `KeyAdapter` , `keyTyped` and `keyPressed` seem to do similar things, but arrow keys never reach `keyTyped` . Read the documentation of `KeyEvent` and explain the difference between a key code and a key character, and between a "key event" and a "typed event". Then explain why `JComponent.WHEN_FOCUSED` , `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT` and `WHEN_IN_FOCUSED_WINDOW` give different results for a key binding in a window with a button, a text field and a game panel. Which one would you use for the game, and which one for a keyboard shortcut inside a text field?

Submission Checklist

Required (the Core exercises, by the next session)

- [ ] `KeyEcho.java` (Exercise 1): the panel shows the name of the last key pressed
- [ ] `MoveSmooth.java` (Exercise 4): both TODOs done; smooth movement, clamped at the walls
- [ ] `MouseFollow.java` and `MouseDrag.java` (Exercise 6): the circle follows the pointer; the rectangle can be dragged and dropped
- [ ] `HitDemo.java` (Exercise 8): both TODOs done; the box turns red only while overlapping the target
- [ ] `CatchDemo.java` (Exercise 10): a catch scores exactly once, a missed egg counts as missed
- [ ] `FallingEggV2.java` (Exercise 11): Pause / Start and the slider work, and the keys keep working after clicking them
- [ ] Homework question 1, level 1 (lives and Game Over)

Extra (finish if you did not in class; optional otherwise)

- [ ] `MoveJerky.java` (Exercise 2) and `FocusDemo.java` (Exercise 3): you observed the pause, reproduced the focus problem and fixed it in both ways
- [ ] `MoveBindings.java` (Exercise 5): the arrows keep working after clicking the button; optional: the smooth version
- [ ] `MouseClick.java` (Exercise 7): both TODOs done; left and right buttons differ
- [ ] `BasketKeys.java` (Exercise 9): the `Basket` class moves smoothly and stays inside the field
- [ ] `ControlMode.java` (Exercise 12, challenge): both TODOs done; keyboard and mouse modes both work
- [ ] Homework question 1, levels 2 and 3, and question 2

Understanding (be ready to explain)

- [ ] Why `KeyListener` has no lambda form, and what `KeyAdapter` is
- [ ] The difference between how the Timer was used in Weeks 3, 4 and 5
- [ ] Why a slider that changes the Timer delay changes the whole game, not only the egg
- [ ] What happens to a held key if the window loses the focus, and how `focusLost` fixes it

Next Week

Week 6 brings many eggs: eggs spawn over time and live in an `ArrayList`. We loop with for-each and `Iterator`, remove caught or missed eggs safely, and add lives and levels.